Fundamentos de PyBullet

Análisis Arquitectónico del Entorno de Simulación

Clase de Robótica

March 26, 2026

Objetivos de la Clase 2

Introducción a Modelos Articulados

En esta sesión dejaremos atrás los objetos estáticos para dar vida a sistemas robóticos complejos. Al finalizar la clase, seremos capaces de:

- **Comprender** el estándar URDF para la descripción de robots.
- **Identificar** la anatomía digital de un robot (Eslabones y Articulaciones).
- **Importar** modelos robóticos reales al entorno de PyBullet.
- **Extraer** telemetría y variables de estado del robot.
- **Aplicar** comandos básicos de control de movimiento a los actuadores.

El Repositorio Integrado: pybullet_data

Modelos Propios de PyBullet

PyBullet incluye una librería auxiliar que contiene decenas de modelos profesionales ya construidos en formato URDF, listos para ser usados en simulación e Inteligencia Artificial.

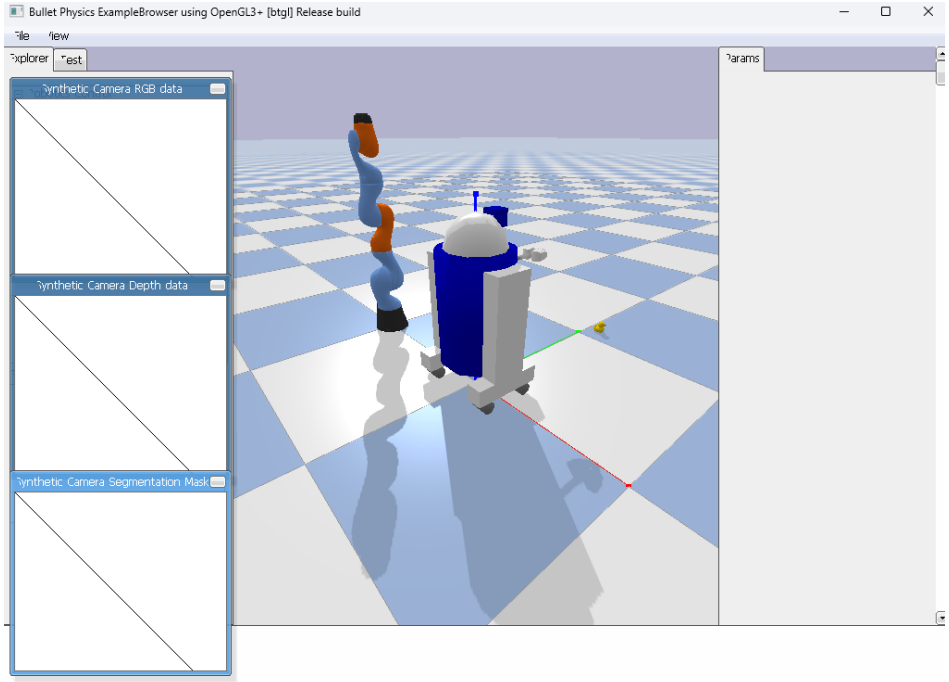


Figure: Modelos Predeterminados de PyBullet

El Repositorio Integrado: pybullet_data

Libreria con datos

```
import pybullet_data
import pybullet as p

# Le decimos al simulador donde buscar estos archivos
p.setAdditionalSearchPath(pybullet_data.getDataPath())
```

Análisis del Código:

- `pybullet_data`: Es como una gran bodega virtual. Contiene mesas, legos, pistas y robots famosos.
- `setAdditionalSearchPath`: Es obligatorio. Si no agregamos esta ruta, cuando le pidamos cargar a "r2d2.urdf", Python buscará en nuestra carpeta de descargas o documentos y el programa fallará lanzando un error.

Cargando un Robot Móvil Clásico (R2D2)

```
# Cargar a R2D2 a medio metro de altura
posicion_r2d2 = [0, 0, 0.5]
r2d2_id = p.loadURDF("r2d2.urdf", posicion_r2d2)
```

Análisis del Código:

- `"r2d2.urdf"`: Como ya configuramos la ruta en el paso anterior, PyBullet sabe exactamente de qué archivo estamos hablando y carga su malla 3D, sus texturas y sus parámetros físicos.
- `posicion_r2d2`: Nuevamente, lo instanciamos en $Z = 0.5$ para que caiga por gravedad hacia el suelo. Si lo ponemos en 0, sus ruedas y patas atravesarán el plano, generando un error en el motor de físicas que hará que el robot salga disparado hacia el cielo.
- `r2d2_id`: Guardamos el número de identificación único de este robot. Si queremos mover su cabeza más adelante, necesitaremos usar este ID.

Cargando un Brazo Industrial (KUKA iiwa)

```
# Cargar un brazo robotico, anclado al piso
posicion_kuka = [-1, 0, 0] # Desplazado 1 metro en X
kuka_id = p.loadURDF("kuka_iiwa/model.urdf",
                     posicion_kuka,
                     useFixedBase=True)
```

Análisis del Código:

- `"kuka_iiwa/model.urdf"`: Noten que aquí indicamos una subcarpeta. Algunos robots complejos vienen en su propia carpeta porque incluyen muchos archivos 3D extra (.STL o .OBJ).
- `useFixedBase=True`: **¡Concepto Crítico!** A diferencia de un robot móvil, un brazo industrial está empernado al piso de la fábrica. Si no agregamos este parámetro, el brazo cederá ante su propio peso (la gravedad) y se desplomará en el suelo apenas inicie la simulación.

Inspeccionando la Complejidad Real

```python
# Cuantas articulaciones tiene R2D2?
num_articulaciones = p.getNumJoints(r2d2_id)
print("Articulaciones de R2D2:", num_articulaciones)
```

Análisis del Código:

- Si ejecutamos esto, la consola nos dirá que R2D2 tiene **15 articulaciones**. El KUKA tiene **7**.
- Esto incluye las ruedas, la cabeza giratoria, y hasta sensores internos modelados como articulaciones fijas.
- Modelar 15 eslabones y 15 articulaciones usando cilindros en un solo script de Python sería una locura. **Por esto mismo, ahora aprenderemos a construir nuestro propio archivo URDF desde cero.**

¿Por qué necesitamos modelos externos (URDF)?

Separar el diseño de la lógica de control

En la Clase 1 creamos formas primitivas directamente en el script de Python. Sin embargo, programar un robot móvil completo línea por línea en nuestro código principal lo vuelve confuso y difícil de mantener.

Conceptos Clave:

- **Estandarización:** URDF (Unified Robot Description Format) es el estándar de la industria. Permite definir la anatomía completa de un robot en un archivo de texto independiente.
- **Independencia del CAD:** Aunque en la industria se usan programas de diseño 3D complejos, en su núcleo, un URDF es solo un archivo estructurado (XML).
- **Nuestra Solución:** Construiremos nuestros propios robots escribiendo el código URDF a mano, utilizando formas primitivas (cajas, cilindros, esferas) para modelar piezas reales como chasis y ruedas.

El Estándar de la Industria: ¿Qué es el URDF?

Unified Robot Description Format

Antes de programar movimientos, necesitamos un modelo. En robótica moderna, no se programa la geometría de un robot directamente en el código de control (Python o C++). Se utiliza un archivo de descripción llamado URDF.

¿Por qué es vital aprenderlo?

- **El Lenguaje Universal:** Es el formato estándar adoptado por la industria a nivel mundial para describir la cinemática y dinámica de cualquier robot.
- **Integración con ROS:** Si en el futuro trabajan con ROS, el URDF es exactamente el mismo formato que usarán para visualizar y controlar robots reales.
- **Modularidad:** Permite separar al ingeniero mecánico/electrónico (que diseña el URDF) del ingeniero de control o IA (que programa el comportamiento en PyBullet).

Anatomía Robótica: Los Eslabones (Links)

Las partes físicas de nuestro robot

Imaginemos un brazo robótico o un auto. Cada pieza sólida, rígida e indeformable se denomina **Link** (Eslabón). Para que el simulador entienda un Link, debemos definirle tres propiedades matemáticas independientes:

- **Propiedad Visual (`visual`):** Es la "piel" del robot. Lo que ven nuestros ojos en la pantalla. Puede ser un cilindro simple o una malla 3D hiperrealista. No afecta la física.
- **Propiedad de Colisión (`collision`):** Es el "esqueleto táctil". Le dice al motor de físicas cuándo el objeto choca con el piso o con otros objetos. Suele ser una geometría más simple que la visual para ahorrar cálculos en el procesador.
- **Propiedad Inercial (`inertial`):** Es la "masa". Define cuánto pesa el eslabón y dónde está su centro de gravedad. Sin esto, la gravedad no sabría cómo afectar a la pieza.

Anatomía Robótica: Las Articulaciones (Joints)

El pegamento cinemático

Si tenemos dos Links (ej. un chasis y una rueda), flotarán en el espacio de forma independiente. Para unirlos y darles movimiento relativo, usamos un **Joint** (Articulación).

Conceptos Clave de un Joint:

- **Relación Padre-Hijo (Parent/Child):** La robótica se modela como un árbol genealógico. El chasis suele ser el "Padre" absoluto. La rueda es el "Hijo". Si el Padre se mueve, arrastra al Hijo. Si el Hijo se mueve (gira), el Padre no necesariamente lo hace.
- **Grados de Libertad (DOF):** El Joint define qué tipo de movimiento está permitido. ¿Se desliza como un pistón (Prismático)? ¿Gira como una bisagra (Revolutivo)? ¿Gira infinitamente como un neumático (Continuo)?
- **Origen y Eje:** El Joint debe definir exactamente en qué coordenada espacial (X, Y, Z) se conectan ambas piezas y sobre qué eje rotarán.

El Estándar URDF

Unified Robot Description Format

El **URDF** es un formato de archivo basado en XML que se ha convertido en el estándar de la industria (especialmente gracias a ROS - Robot Operating System) para describir la cinemática, dinámica y geometría de un robot.

¿Qué contiene exactamente un archivo URDF?

- **Geometría Visual:** Cómo se ve el robot (archivos .STL, .DAE, o colores).
- **Geometría de Colisión:** Formas simplificadas para que el motor físico calcule los choques sin sobrecargar la CPU.
- **Propiedades Físicas:** Masa, fricción, inercia de cada pieza.
- **Estructura Cinemática:** Cómo están conectadas las piezas entre sí.

Anatomía de un Robot: Links y Joints

Construyendo la Cadena Cinemática

Para PyBullet (y para cualquier motor físico), un robot no es una sola pieza, sino un conjunto de partes conectadas en forma de árbol.

Definiciones Fundamentales:

- **Link (Eslabón):** Es un cuerpo rígido. Representa las partes "sólidas" del robot (ej. el antebrazo, una rueda, el chasis). Cada link tiene su propio centro de masa y propiedades de colisión.
- **Joint (Articulación):** Es la conexión mecánica que une a dos Links. Define *cómo* se pueden mover esos dos eslabones entre sí.
- **Relación Padre-Hijo:** Un Joint conecta un "Link Padre" con un "Link Hijo", creando una jerarquía desde la base del robot hasta su efector final.

Tipos de Articulaciones (Joints)

Grados de Libertad (DOF)

Dependiendo del diseño del robot, las articulaciones restringen el movimiento de diferentes maneras. En PyBullet, las más comunes son:

- **Revolute (Revolutiva):** Gira sobre un solo eje y tiene límites de ángulo (ej. la rodilla de un robot cuadrúpedo o un codo humano).
- **Continuous (Continua):** Igual que la revolutiva, pero sin límites de ángulo. Puede girar infinitamente (ej. las ruedas de un rover).
- **Prismatic (Prismática):** Se desliza a lo largo de un solo eje de forma lineal, también con límites (ej. un pistón hidráulico o una impresora 3D).
- **Fixed (Fija):** Une dos eslabones rígidamente. No permite ningún movimiento relativo (0 grados de libertad). Útil para ensamblar sensores al chasis.

Traduciendo la Teoría a Código: Estructura XML

```
<?xml version="1.0"?>
<robot name="mi_primer_robot">

    </robot>
```

Análisis del Código:

- Un archivo URDF se escribe en **XML**, un lenguaje de etiquetas para estructurar datos (no en Python).
- La línea `<?xml version="1.0"?>` indica el tipo de documento.
- `<robot name="...">` es la etiqueta principal que contiene todo el modelo.
- Los comentarios en XML se escriben como `<!-- comentario -->` y el simulador los ignora.

Programando un Link: El Chasis (Paso a Paso)

```
<link name="chasis">
    <visual>
        <geometry>
            <box size="0.4 0.2 0.1"/>
        </geometry>
    </visual>
</link>
```

Análisis del Código:

- `<link name="chasis">`: Define una pieza física llamada "chasis".
- `<visual>`: Indica cómo se verá en la simulación.
- `<geometry>`: Especifica la forma geométrica.
- `<box size="0.4 0.2 0.1"/>`: Crea una caja de $0.4 \times 0.2 \times 0.1$ metros ($40 \times 20 \times 10$ cm). En robótica, las unidades son en metros.

Programando un Joint: Uniendo piezas (Paso a Paso)

```
<joint name="union_rueda" type="continuous">
    <parent link="chasis"/>
    <child link="rueda_delantera"/>
    <origin xyz="0.15 0.1 0" rpy="1.5708 0 0"/>
    <axis xyz="0 0 1"/>
</joint>
```

Análisis del Código:

- `type="continuous"`: Define una unión de giro infinito (como una rueda).
- `parent` y `child`: Indican qué pieza es base (chasis) y cuál se mueve (rueda).
- `origin xyz="0.15 0.1 0"`: Posición de la rueda respecto al centro del chasis (en metros).
- `origin rpy="1.5708 0 0"`: Rotación inicial en Roll-Pitch-Yaw. 1.5708 rad 90°.
- `axis xyz="0 0 1"`: Eje sobre el cual gira la rueda (eje Z).

Construyendo el Robot

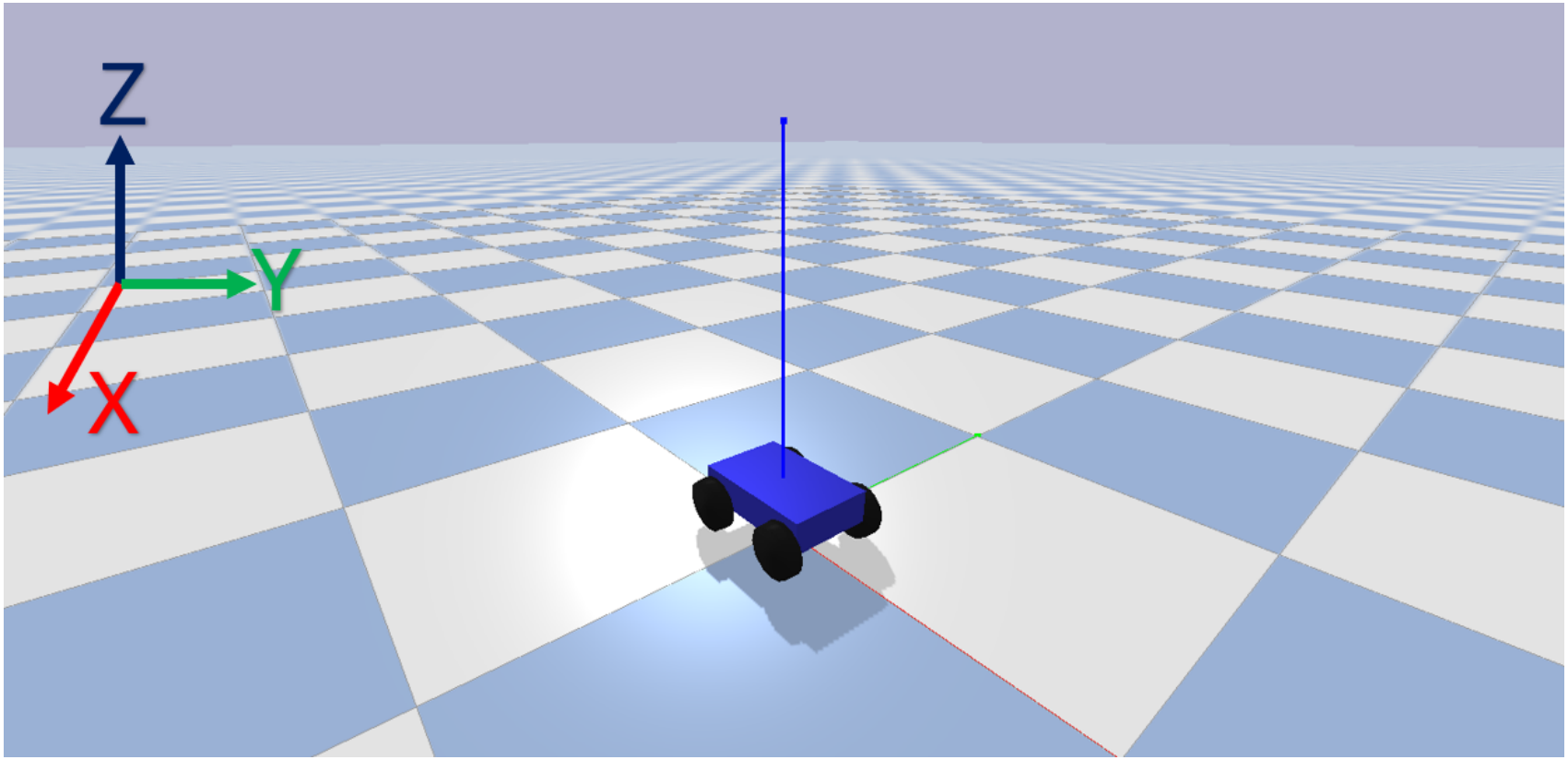


Figure: Robot a construir

Construyendo el Robot: Materiales y Colores

```xml
<?xml version="1.0"?>
<robot name="robot_movil_14kg">

    <material name="azul">
        <color rgba="0.2 0.2 0.8 1"/>
    </material>
    <material name="negro">
        <color rgba="0.1 0.1 0.1 1"/>
    </material>
```

Análisis del Código:

- Definimos el nombre general del robot.
- `<material>`: Declaramos los colores que usaremos en el modelo.
- `rgba`: Red, Green, Blue y Alpha (transparencia), con valores entre 0 y 1. Permite diferenciar visualmente las piezas.

El Eslabón Principal (Chasis): Geometría

```
<link name="chasis">
    <visual>
        <origin xyz="0 0 0" rpy="0 0 0"/>
        <geometry>
            <box size="0.4 0.25 0.1"/>
        </geometry>
        <material name="azul"/>
    </visual>
    <collision>
        <origin xyz="0 0 0" rpy="0 0 0"/>
        <geometry>
            <box size="0.4 0.25 0.1"/>
        </geometry>
    </collision>
```

Análisis del Código:

- `<link name="chasis">`: Define el cuerpo principal del robot.
- `<origin>`: Ubica la geometría en el origen (0,0,0) del eslabón.
- `<box size="...">`: Dimensiones en metros (largo X, ancho Y, alto Z).
- Se repite la geometría en `<visual>` y `<collision>` y se aplica el material definido.

El Eslabón Principal (Chasis): Física e Inercia

```
    <inertial>
        <origin xyz="0 0 0" rpy="0 0 0"/>
        <mass value="10.0"/>
        <inertia ixx="0.0604" ixy="0.0" ixz="0.0"
                 iyy="0.1416" iyz="0.0"
                 izz="0.1854"/>
    </inertial>
</link>
```

Análisis del Código:

- `<mass value="10.0"/>`: Asignamos un peso de 10 kg al chasis.
- `<inertia>`: Esta matriz es el secreto para una simulación realista. Le dice al motor de físicas cuánta resistencia opone la caja a rotar sobre sus propios ejes.
- Los valores I_{xx}, I_{yy}, I_{zz} se calcularon con las fórmulas reales de dinámica para un prisma rectangular sólido. Si dejamos esto en cero, la caja reaccionaría a los choques como si estuviera hecha de plumavit o papel.

Modelando una Rueda: El Cilindro

```
<link name="rueda_delantera_derecha">
    <visual>
        <origin xyz="0 0 0" rpy="0 0 0"/>
        <geometry>
            <cylinder length="0.04" radius="0.08"/>
        </geometry>
        <material name="negro"/>
    </visual>
    <inertial>
        <mass value="1.0"/>
        <inertia ixx="0.0017" ixy="0.0" ixz="0.0"
                 iyy="0.0017" iyz="0.0" izz="0.0032"/>
    </inertial>
</link>
```

Análisis del Código:

- `<cylinder>`: Define un cilindro de 4 cm de largo y 8 cm de radio.
- Se asigna 1 kg de masa y su inercia correspondiente.
- Por defecto, el cilindro apunta en el eje Z; la rotación se ajusta en la articulación.

La Articulación (Joint): Posición y Rotación

```
<joint name="joint_rueda_dd" type="continuous">
    <parent link="chasis"/>
    <child link="rueda_delantera_derecha"/>
    <origin xyz="0.15 -0.15 -0.05" rpy="-1.5708 0 0"/>
    <axis xyz="0 0 1"/>
</joint>
```

Análisis del Código:

- `type="continuous"`: Permite giro infinito entre chasis (padre) y rueda (hijo).
- `xyz="0.15 -0.15 -0.05"`: Posición de la rueda respecto al chasis (en metros).
- `rpy="-1.5708 0 0"`: Rotación de 90° en X para colocar el cilindro como neumático.
- `axis xyz="0 0 1"`: Eje local sobre el que gira la rueda.

Masa vs. Momento de Inercia

La resistencia al movimiento rotacional

En física lineal, la **masa** (m) nos dice cuánta resistencia opone un objeto a ser empujado (traslación). En dinámica rotacional, el **momento de inercia** (I) nos dice cuánta resistencia opone un objeto a *girar* sobre un eje.

Conceptos Clave:

- **Distribución de la masa:** Dos ruedas pueden pesar 1 kg, pero si una es un disco macizo y la otra es un anillo (como una rueda de bicicleta), la energía necesaria para hacerlas girar será totalmente distinta.
- **Dependencia del eje:** Girar una regla de plástico sobre su eje longitudinal es fácil. Girarla como si fuera la hélice de un helicóptero cuesta más. El momento de inercia cambia dependiendo de qué eje usemos para rotar.

El Tensor de Inercia en URDF

La matriz 3x3 de rotación

Para que PyBullet calcule cómo choca, rebota y gira nuestro robot en el espacio 3D, requiere un Tensor de Inercia, representado por una matriz simétrica de 3x3.

```
<inertia ixx="..." ixy="0.0" ixz="0.0"
         iyy="..." iyz="0.0"
         izz="..."/>
```

Análisis del Código:

- **Momentos Principales (I_{xx}, I_{yy}, I_{zz}):** Representan la inercia cuando el objeto gira exactamente sobre sus ejes X, Y o Z. Son los números que debemos calcular.
- **Productos de Inercia (I_{xy}, I_{xz}, I_{yz}):** Representan asimetrías en la distribución de la masa. Como usamos formas primitivas perfectas y simétricas (cajas y cilindros) con masa uniforme, estos valores siempre serán **cero**.

Cálculo Real: El Chasis (Caja Sólida)

Matemática detrás del URDF

Para una caja sólida de masa m, con dimensiones de largo (d), ancho (w) y alto (h), las fórmulas de los momentos principales son:

$$I_{xx} = \frac{1}{12}m(h^2 + w^2) \quad I_{yy} = \frac{1}{12}m(h^2 + d^2) \quad I_{zz} = \frac{1}{12}m(w^2 + d^2)$$

Aplicado a nuestro robot de 10 kg ($d = 0.4, w = 0.25, h = 0.1$):

- $I_{xx} = \frac{10}{12} \times (0.1^2 + 0.25^2) = 0.833 \times 0.0725 = \mathbf{0.0604}$
- $I_{yy} = \frac{10}{12} \times (0.1^2 + 0.4^2) = 0.833 \times 0.1700 = \mathbf{0.1416}$
- $I_{zz} = \frac{10}{12} \times (0.25^2 + 0.4^2) = 0.833 \times 0.2225 = \mathbf{0.1854}$

¡De aquí salieron exactamente los números que pusimos en el código XML!

Cálculo Real: Las Ruedas (Cilindro Sólido)

Matemática detrás del URDF

Para un cilindro sólido de masa m, radio r y longitud/grosor h, rotando sobre sus diferentes ejes:

$$I_{xx} = I_{yy} = \frac{1}{12} m(3r^2 + h^2)$$

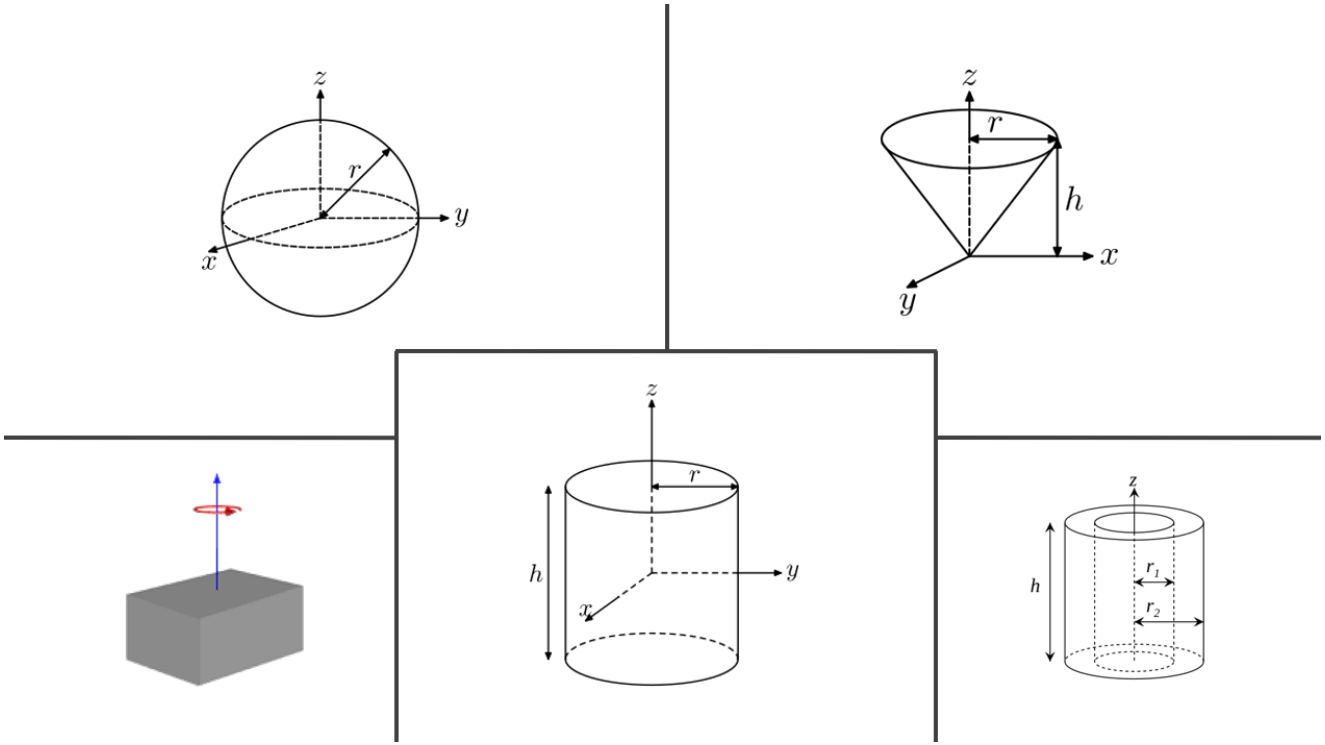
$$I_{zz} = \frac{1}{2} m r^2$$

Aplicado a nuestras ruedas de 1 kg ($r = 0.08, h = 0.04$):

- El eje Z es el eje natural del cilindro (el eje de la rueda).
- $I_{zz} = 0.5 \times 1 \times (0.08^2) = 0.5 \times 0.0064 = \mathbf{0.0032}$
- $I_{xx} = I_{yy} = \frac{1}{12} \times 1 \times (3 \times 0.0064 + 0.0016) = \mathbf{0.0017}$

Nota: Si se ingresan valores de inercia muy erróneos, verán que el robot en PyBullet vibra incontrolablemente o requiere torques gigantescos para moverse.

Link con las fórmulas



Link Fórmulas